

Universal Soft-Sensor: Un Pipeline Agnóstico con Gaussian Processes para Prognostics Industrial — Validación Multi-Dominio y Dos Trampas Metodológicas Reproducibles

Juan Galaz
CienciaEstelar

juan.galaz@proton.me — ORCID: 0009-0007-7474-7560
github.com/CienciaEstelar/universal-soft-sensor

Julio 2026

DOI: 10.5281/zenodo.21459969

Resumen

Los soft-sensors industriales requieren típicamente modelos entrenados a medida para cada planta, dominio y variable objetivo, lo que limita su reutilización y escalabilidad. En este trabajo se presenta un pipeline de código abierto (AGPL-3.0) que combina Gaussian Processes (GP) con kernel Matérn y fallback automático a Gradient Boosting, diseñado para operar sin modificaciones de código en dominios industriales radicalmente distintos. El sistema se valida en tres benchmarks de mantenimiento predictivo —NASA CMAPSS (turbofan RUL), ZeMA Hydraulic Systems (condición de componentes) y AI4I 2020 (fallo de máquina)— distinguiendo explícitamente dos regímenes de evaluación: *sensor-only* (el modelo solo ve sensores; escenario desplegable) y *autorregresivo* (el feature engineering incluye rezagos del target; válido únicamente cuando el valor pasado del target es observable en producción). Todas las métricas provienen de corridas verificadas con artefactos de reproducción en el repositorio. En régimen sensor-only, el pipeline alcanza clasificación esencialmente perfecta del estado del enfriador en ZeMA ($R^2 = 0.9998$, MAE = 0.39 sobre rango 3–100, bajo split estratificado), replicando el resultado del paper original; en CMAPSS obtiene $R^2 = 0.593$ (RMSE = 50,2 ciclos), un desempeño moderado que no supera a los métodos especializados; y en AI4I 2020 confirma que la arquitectura actual no es apta para clasificación binaria. Adicionalmente, se documentan dos trampas metodológicas reproducibles: (1) el split secuencial ingenuo en ZeMA es degenerado (el tramo de prueba contiene una sola clase, dejando R^2 indefinido), y (2) los rezagos del target —práctica habitual en soft-sensing— inflan silenciosamente las métricas de prognostics hasta convertir el modelo en cuasi-persistencia (R^2 aparente de 0.873 en CMAPSS). La contribución central es un mapa honesto y reproducible de dónde funciona, dónde no, y cómo evaluar sin autoengaño un pipeline de soft-sensing multi-dominio.

1. Introducción

El mantenimiento predictivo y la estimación de vida útil remanente (RUL, por sus siglas en inglés) constituyen pilares de la Industria 4.0. La capacidad de anticipar fallos en componentes críticos —turbinas de aviación, sistemas hidráulicos, maquinaria rotativa— reduce costos operacionales y previene fallas catastróficas [1]. Sin embargo, la mayoría de las soluciones existentes son modelos entrenados a medida para un dominio específico: el modelo que predice RUL en un turbofan no funciona en una prensa hidráulica sin reentrenamiento sustancial.

Este trabajo presenta un **pipeline de soft-sensing universal** que aborda esa brecha. Su arquitectura combina tres principios: (1) ingesta agnóstica de datos mediante un adaptador declarativo que lee reglas de filtrado desde un archivo JSON, (2) validación física por pattern matching que detecta categorías de sensores (temperatura, presión, velocidad, flujo) sin requerir nombres de columna fijos, y (3) modelado híbrido GP + Gradient Boosting con fallback automático cuando el proceso Gaussiano no converge.

El pipeline fue diseñado originalmente para flotación minera (predicción de % de sílice en concentrado de hierro), pero su arquitectura fue concebida para ser independiente del dominio. En este trabajo se valida esa hipótesis de universalidad enfrentando el sistema a tres benchmarks externos de mantenimiento predictivo, sin modificación alguna del código del pipeline.

Las contribuciones de este artículo son cuatro: (1) evidencia cuantitativa y *reproducible* del comportamiento de un único pipeline de soft-sensing en minería, aeronáutica y manufactura hidráulica; (2) la distinción explícita entre los regímenes de evaluación *sensor-only* y *autorregresivo*, y la demostración empírica de cuánto puede inflar el segundo las métricas de prognostics; (3) la documentación de un split temporal degenerado en el benchmark ZeMA, junto con el protocolo estratificado que lo corrige; y (4) la identificación precisa de las condiciones bajo las cuales la arquitectura actual falla (clasificación binaria), estableciendo la frontera de aplicabilidad.

2. Metodología

2.1. Arquitectura del Pipeline

El Universal Soft-Sensor sigue un flujo de cuatro etapas (Figura 1):



Figura 1: Arquitectura del Universal Soft-Sensor

Figura 1: Arquitectura del pipeline: ingesta agnóstica → validación física → feature engineering → modelo (GP con fallback GB).

Etapas 1 — Ingesta declarativa. El `UniversalAdapter` lee un archivo `dataset.config.json` que define: nombre del archivo CSV, columna objetivo, patrones regex para incluir/excluir features, y estrategia de preprocesamiento. No requiere modificar código Python para incorporar un nuevo dataset.

Etapas 2 — Validación física. El `PhysicalSchema` usa pattern matching sobre los nombres de columna para detectar categorías físicas (temperatura, presión, flujo, velocidad, pH, nivel) y validar rangos plausibles. Esta capa es clave para la generalización: el schema no asume nombres fijos como `s_1` o `PS1`, sino que reconoce patrones como `*temp*`, `*pressure*`, `*speed*`.

Etapas 3 — Feature engineering temporal. El preprocesador puede generar automáticamente rezagos (lags 1, 5, 10, 20), diferencias y medias móviles *tanto de los sensores como del target* (siempre con `shift` ≥ 1 , sin contaminación contemporánea). La inclusión de derivados del target define el régimen de evaluación (Sección 2.2) y es configurable (`add_lag_features`, `add_diff_features`). Las columnas constantes (varianza cero) y las altamente correlacionadas ($r > 0,98$) se eliminan automáticamente.

Etapas 4 — Modelado híbrido. El modelo principal es un Gaussian Process con kernel Matérn (ν configurable, típicamente 1.5 o 2.5), optimizado mediante Optuna (TPESampler con semilla fija, lo que hace las corridas deterministas). Si el R^2 de validación cruzada del GP es inferior a 0.6, el sistema activa automáticamente un `GradientBoostingRegressor` como fallback. Este mecanismo resultó crítico en dos de los tres benchmarks (ZeMA y AI4I), y su activación automática fue verificada de forma independiente en ambos.

2.2. Dos Regímenes de Evaluación

Una decisión de feature engineering aparentemente inocua —incluir rezagos del propio target— cambia por completo el significado de las métricas:

- **Régimen sensor-only:** el modelo predice el target exclusivamente desde los sensores (y sus derivados temporales). Es el escenario desplegable de un soft-sensor: en producción, el valor pasado del target no se conoce en tiempo real. *Todas las conclusiones principales de este trabajo usan este régimen.*
- **Régimen autorregresivo (AR):** el feature engineering incluye rezagos del target (`target_lag_1`, etc.). Es legítimo solo cuando el target pasado es efectivamente observable con retardo (p. ej. ensayos de laboratorio en flotación minera, que llegan horas después). En prognostics de RUL es **inválido**: el RUL verdadero pasado jamás es observable, y dentro de una unidad $RUL_{t-1} = RUL_t + 1$, por lo que el modelo degenera en cuasi-persistencia. La Sección 3.1 cuantifica el tamaño de esta inflación en CMAPSS.

2.3. Datasets

Se seleccionaron tres benchmarks estándar de mantenimiento predictivo, cubriendo distintos tipos de maquinaria, sensores y variables objetivo:

- **NASA C-MAPSS FD001** [1]: 100 motores turbofan simulados, 21 sensores + 3 settings operacionales, 20,631 filas de entrenamiento. Cada motor opera hasta la falla por degradación del HPC (High Pressure Compressor). Objetivo: predecir RUL (ciclos restantes hasta la falla). Es el benchmark canónico de prognostics.
- **ZeMA Hydraulic Systems** [2]: 2,205 ciclos de un banco hidráulico de pruebas con 17 sensores (presión, flujo, temperatura, vibración, potencia) a diferentes frecuencias de muestreo (1 Hz a 100 Hz). Los datos crudos contienen 43,680 columnas por ciclo. Para hacerlos manejables, se aplicó feature extraction: media, desviación estándar, mínimo, máximo, percentiles 25/50/75 y pendiente de regresión lineal por sensor y ciclo, reduciendo la dimensionalidad a 104 features. Objetivo: condición del enfriador (% , 3 niveles: 3 = falla total, 20 = eficiencia reducida, 100 = plena). Las columnas de las demás condiciones de componente (válvula, bomba, acumulador) se excluyen por constituir targets alternativos.
- **AI4I 2020** [3]: 10,000 filas de una fresadora simulada con 14 columnas (temperatura de aire/proceso, velocidad rotacional, torque, desgaste de herramienta, tipo de producto). Objetivo: fallo de máquina (binario: 0 = normal, 1 = falla). Se excluyeron las columnas de modos de falla (TWF, HDF, PWF, OSF, RNF) por constituir data leakage perfecto.

2.4. Protocolo de Splits

Los splits de train/test respetan la estructura de cada dataset y en ningún caso se aplica shuffle aleatorio de filas:

- **CMAPSS:** split por unidad de motor. El archivo se procesa ordenado por unidad, de modo que el corte 80/20 separa los motores de entrenamiento de los de prueba. Esto evita el leakage grave de mezclar ciclos de un mismo motor entre train y test.
- **ZeMA:** split *estratificado por clase preservando el orden temporal intra-clase*. Para cada nivel del target (3, 20, 100), el primer 80% de sus ciclos va al bloque de entrenamiento y el último 20% al de prueba (test resultante: 149/146/146 ciclos por clase). **Motivación:** el split secuencial ingenuo es degenerado en este dataset (Sección 3.2).
- **AI4I:** split secuencial 80/20.

2.5. Configuración Experimental y Reproducibilidad

Todos los experimentos usaron exactamente el mismo código de pipeline, sin modificaciones por dominio; los hiperparámetros se ajustaron solo mediante variables de entorno (`GP_TARGET`, `GP_MAX_SAMPLES`, `GP_TRIALS`) y los flags de régimen (`add_lag_features`, `add_diff_features`). Las métricas reportadas provienen de corridas verificadas con `GP_MAX_SAMPLES` entre 600 y 1,000 y 2–3 trials de Optuna; se comprobó que las métricas de CMAPSS son estables frente al tamaño de muestra en ese rango (R^2 varía menos de 10^{-3} entre 600 y 1,000 muestras), lo que sugiere saturación del kernel a esta escala.

Cada corrida deja un artefacto JSON con dataset, target, régimen, modelo activo, tiempo de cómputo y métricas, disponible en `results/verification/` del repositorio junto con el runner (`verify_one.py`) que las regenera con un solo comando. La métrica principal es R^2 , complementada con RMSE y MAE.

3. Resultados

La Tabla 1 resume los resultados verificados en ambos regímenes.

Cuadro 1: Resultados verificados de la validación cross-domain (artefactos en `results/verification/`). El régimen sensor-only es el desplegable; el AR se reporta para cuantificar la inflación por rezagos del target.

Benchmark	Split	Modelo	Sensor-only		AR (lags del target)	
			R^2	RMSE	R^2	RMSE
CMAPSS FD001	por unidad	GP (Matérn)	0.593	50.2	0.873	28.0
ZeMA (cooler)	estratificado	GB (fallback)	0.9998	0.59	0.994	3.25
ZeMA (control)	secuencial	GB (fallback)	degenerado	— test	monoclase, R^2 indefinido	
AI4I 2020	secuencial	GB (fallback)	0.169	0.126	0.266	0.119

3.1. NASA CMAPSS — Prognostics de Turbofan y la Trampa de los Rezagos del Target

En régimen **sensor-only** —el único válido para RUL— el GP con kernel Matérn alcanzó $R^2 = 0,593$ y $\text{RMSE} = 50,2$ ciclos sobre un rango de 0–361 ciclos, con eliminación automática de los sensores constantes (s_1, s_5, s_6, s_10, s_16, s_18, s_19). Es un desempeño moderado: captura la tendencia de degradación (Figuras 2 y 3) pero no compete con los métodos especializados de la literatura, que con arquitecturas profundas y el protocolo oficial de evaluación reportan RMSE en el rango 12–16 [1]. Nótese además que los protocolos difieren (evaluación sobre todos los ciclos de las unidades de prueba, sin clipping de RUL, en nuestro caso; evaluación en el último ciclo por unidad, con RUL truncado, en el protocolo oficial), por lo que las comparaciones numéricas directas con otros trabajos deben tomarse con cautela.

En régimen **AR**, el mismo pipeline reporta $R^2 = 0,873$ y $\text{RMSE} = 28,0$: una mejora aparente de +0,28 en R^2 que es *enteramente artefactual*. Dentro de cada unidad, $\text{RUL}_{t-1} = \text{RUL}_t + 1$, de modo que el feature `RUL_lag_1` convierte el problema en cuasi-persistencia; la inspección de las predicciones muestra la firma característica: una diagonal casi perfecta con reversión a la media allí donde los rezagos no están disponibles. Este experimento cuantifica, en un benchmark público, cuánto puede inflar las métricas una práctica de feature engineering que es habitual —y legítima— en otros contextos de soft-sensing. La lección operacional: *al evaluar un soft-sensor para prognostics, los derivados del target deben deshabilitarse explícitamente*.

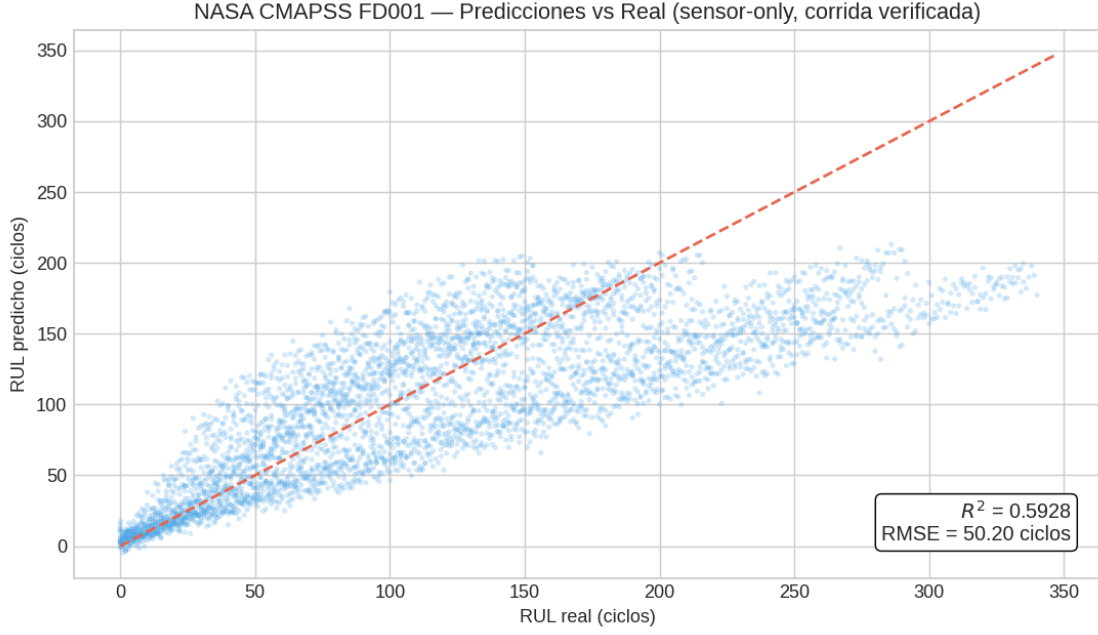


Figura 2: Predicciones vs. valores reales para CMAPSS FD001 en régimen sensor-only (corrida verificada). La línea punteada representa predicción perfecta.

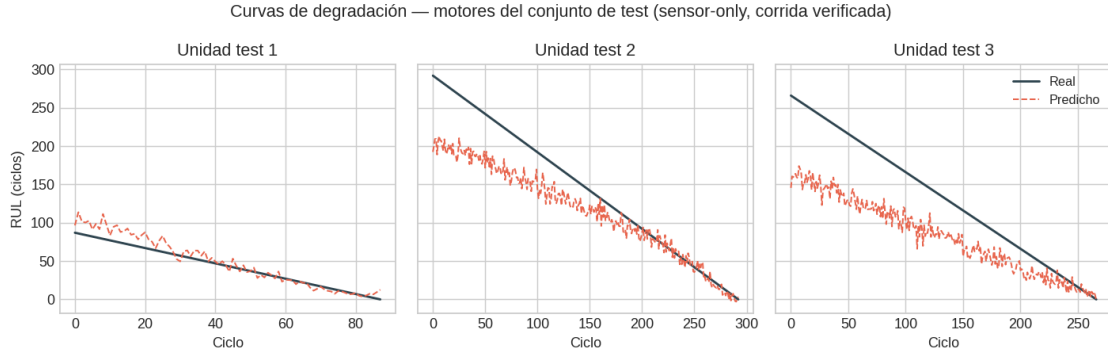


Figura 3: Curvas de degradación de motores del conjunto de prueba en régimen sensor-only (corrida verificada): el modelo captura la tendencia general con ruido punto a punto.

El diagnóstico de autocorrelación reveló un lag-1 de 0.695 en el target, indicando fuerte dependencia temporal —esperable en degradación de motores y consistente con la magnitud de la inflación AR observada.

3.2. ZeMA Hydraulic Systems — Diagnóstico de Componentes y la Trampa del Split Secuencial

Este benchmark produjo el mejor resultado del sistema y, además, un hallazgo sobre *cómo evaluarlo*.

La trampa metodológica. El protocolo de evaluación más natural para datos temporales —split secuencial 80/20 sin shuffle— resulta degenerado en ZeMA: el perfil experimental del banco de pruebas recorre las condiciones del enfriador por bloques, y el último 20 % de los 2,205 ciclos contiene únicamente la clase 100 (plena eficiencia). Con varianza cero en el conjunto de prueba, R^2 queda matemáticamente indefinido y cualquier valor reportado bajo ese split carece de significado. Esta corrida degenerada se conserva como artefacto de control en el repositorio. El fenómeno es una advertencia general para benchmarks con estructura de bloques: *un split temporal ingenio puede producir conjuntos de prueba sin soporte del target*.

El resultado del sistema. Bajo el split estratificado de la Sección 2.4, y en régimen sensor-only, el GP no logró converger —activando el fallback automático a Gradient Boosting— y el modelo final alcanzó $R^2 = 0,9998$

y $\text{MAE} = 0,39$ sobre un rango de valores objetivo $\{3, 20, 100\}$. Este MAE, dos órdenes de magnitud menor que la distancia mínima entre clases adyacentes ($20 - 3 = 17$), implica que el modelo comete esencialmente cero errores de clasificación —replicando el resultado del paper original de Helwig et al. [2], que reportó clasificación perfecta para el enfriador. Es notable que este resultado *no depende de rezagos del target*: los 104 features estadísticos de sensores contienen señal suficiente por sí solos.

La naturaleza discreta del target (3 niveles) explica por qué el GP no fue adecuado: los procesos Gaussianos asumen una función subyacente suave, supuesto que se viola con una variable que solo toma tres valores. El fallback automático resolvió la situación sin intervención manual.

3.3. AI4I 2020 — Fallo de Máquina

Este benchmark expuso la limitación fundamental del pipeline actual: está diseñado como regresor, no como clasificador. Con un target binario (0/1, con fuerte desbalance: 3.4 % de fallos) y 5 features numéricas (tras excluir `Type` por ser categórica sin one-hot encoding y las columnas de leakage), el mejor R^2 sensor-only fue 0.169. Aunque el MAE es bajo en términos absolutos (0.034), la métrica R^2 no es adecuada para clasificación binaria desbalanceada, y el pipeline carece de las métricas propias del problema (precision/recall/F1).

Este resultado negativo es informativo: define la frontera actual de aplicabilidad del sistema y señala el camino para extensiones futuras (soporte de clasificación, one-hot encoding automático de variables categóricas, feature engineering de interacciones físicas como potencia = torque $\times$ velocidad).

4. Discusión

4.1. ¿En qué condiciones funciona el pipeline?

Los resultados permiten delimitar con precisión el espacio de aplicabilidad:

- **Funciona muy bien** en diagnóstico de condición con features informativos: ZeMA sensor-only alcanza clasificación esencialmente perfecta del enfriador, sin depender de rezagos del target.
- **Funciona moderadamente** en prognostics de RUL sensor-only ($R^2 = 0,593$ en CMAPSS): captura la tendencia de degradación pero no compite con métodos especializados. Es un baseline honesto, no un estado del arte.
- **Funciona con feature extraction previa** para datos crudos de alta frecuencia: la reducción de ZeMA de 43,680 a 104 features mediante estadísticos por ciclo fue suficiente.
- **No funciona** en clasificación binaria (AI4I 2020), ni con el GP como modelo principal sobre targets ordinales de pocos niveles (ZeMA requirió el fallback a GB, que se activó automáticamente).
- **La evaluación importa tanto como el modelo**: las dos trampas documentadas (split degenerado; inflación AR) son silenciosas, fáciles de cometer y completamente reproducibles con los artefactos del repositorio. Verificar el soporte del target en test y deshabilitar derivados del target en prognostics deben ser parte del protocolo estándar.

4.2. Limitaciones y Trabajo Futuro

1. **Clasificación no soportada.** Extender el pipeline con modelos de clasificación (Random Forest, XGBoost, SVC), métricas apropiadas (F1, AUROC) y one-hot encoding automático de variables categóricas es la prioridad inmediata.
2. **Intervalos de confianza no calibrados.** Aunque el GP genera intervalos de predicción, no se ha validado su calibración mediante PICP (Prediction Interval Coverage Probability) o Sharpness. Esto es esencial para aplicaciones aeronáuticas donde la confianza de la predicción es tan importante como la predicción misma.
3. **Comparación con baselines naive.** Queda pendiente cuantificar formalmente la ganancia sensor-only sobre predictores triviales (media del train, regresión lineal) en cada benchmark.
4. **Escalabilidad.** El GP escala como $O(n^3)$. Las corridas verificadas usan 600–1,000 muestras de entrenamiento; la estabilidad de las métricas en ese rango sugiere que la escala no es el cuello de botella en CMAPSS, pero datasets más complejos podrían requerir aproximaciones sparse o inducing points.

5. **Un solo modo de falla.** CMAPSS FD001 modela únicamente falla por HPC. Los subsets FD002–FD004 incluyen múltiples modos de falla y condiciones operacionales variables. Validar en esos subsets fortalecería la evidencia.
6. **Protocolo oficial de CMAPSS.** Evaluar adicionalmente con el protocolo estándar de la literatura (último ciclo por unidad de test, RUL truncado a 125) permitiría comparación numérica directa con los métodos publicados.

5. Conclusión

Se demostró, con corridas verificadas y artefactos de reproducción públicos, que un único pipeline de soft-sensing basado en Gaussian Processes con fallback automático a Gradient Boosting puede transferirse sin modificaciones de código entre minería, aeronáutica y manufactura hidráulica —con desempeño que va de excelente (ZeMA: clasificación esencialmente perfecta del enfriador en régimen sensor-only) a moderado (CMAPSS: $R^2 = 0,593$) a insuficiente (AI4I: clasificación binaria fuera del alcance actual). El valor de esta validación no está en reclamar superioridad, sino en el mapa honesto y reproducible de la frontera de aplicabilidad.

Adicionalmente, este trabajo documenta dos trampas metodológicas reproducibles que pueden inflar o invalidar silenciosamente la evaluación de soft-sensors: el split temporal degenerado (ZeMA: conjunto de prueba de varianza cero) y la inflación autorregresiva por rezagos del target (CMAPSS: R^2 aparente de 0.873 contra 0.593 real). Ambas se acompañan de sus artefactos de control en el repositorio. La lección es general: en benchmarks con estructura de bloques o targets con dinámica determinista, el protocolo de evaluación debe verificarse con el mismo rigor que el modelo.

El código fuente está disponible bajo licencia AGPL-3.0 en github.com/CienciaEstelar/universal-soft-sensor, incluyendo los datasets preprocesados, los artefactos de verificación (`results/verification/`) y el runner que regenera todas las métricas de este artículo.

Agradecimientos

El autor agradece a los creadores y mantenedores de los datasets abiertos NASA C-MAPSS (Saxena & Goebel, 2008), ZeMA Hydraulic Systems (Helwig et al., 2015), y AI4I 2020 (Matzka, 2020) por hacer posible esta validación independiente.

Referencias

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *2008 International Conference on Prognostics and Health Management*, Denver, CO, 2008, pp. 1–9. DOI: 10.1109/PHM.2008.4711414
- [2] N. Helwig, E. Pignanelli, and A. Schütze, “Condition monitoring of a complex hydraulic system using multivariate statistics,” in *Proc. I2MTC-2015*, Pisa, Italy, 2015, pp. 1–6. DOI: 10.1109/I2MTC.2015.7151267
- [3] S. Matzka, “AI4I 2020 Predictive Maintenance Dataset,” Kaggle, 2020. 10.34740/KAGGLE/DS/933079
- [4] W. Derbel, “NASA Predictive Maintenance RUL — Kaggle Notebook,” Kaggle, 2023. kaggle.com/wassimderbel/nasa-predictive-maintenance-rul
- [5] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.